

Datasets and Models: Acquisition and Usage

1. Datasets

1.1 Emotional Speech Dataset (ESD)

What it is

The Emotional Speech Dataset (ESD) is an open-source emotional speech corpus developed by the National University of Singapore (NUS) and Singapore University of Technology and Design (SUTD). It contains 350 parallel utterances — meaning every speaker says the exact same sentences — recorded by 10 native English speakers and 10 native Mandarin Chinese speakers, covering 5 emotion categories: **Neutral, Happy, Angry, Sad, and Surprise**. More than 29 hours of audio were recorded in a controlled acoustic environment. The dataset is publicly available and free for non-commercial research use.

How to get it

The dataset is hosted on GitHub and must be requested via email:

1. Go to <https://github.com/HLTSingapore/Emotional-Speech-Data>
2. Download the license agreement form from the repository
3. Complete the form and email it to zhoukun@u.nus.edu
4. You will receive a Google Drive download link

The English speakers are numbered **0011 to 0020**. Each speaker folder contains subfolders per emotion:

```
ESD/  
  0011/  
    Angry/  
      0011_000351.wav  
      0011_000352.wav  
      ...  
    Happy/  
    Neutral/  
    Sad/  
    Surprise/  
  0012/  
    ...  
  0020/
```

Each wav file is recorded at **22,050 Hz, mono, 16-bit PCM**.

How we used it

Role — Reference audio source for XTTS-v2

XTTS-v2 conditions synthesis on a short reference audio clip to clone the voice and emotional style of that clip. We selected speaker **0011** as our reference speaker because their recordings were consistent in quality across all five emotion categories.

For each emotion, we concatenated multiple short utterances from speaker 0011 into a single reference clip of at least 8 seconds, because XTTS-v2 recommends a minimum of 6 seconds for reliable voice cloning conditioning. The concatenation was done with this script:

```
import soundfile as sf
import numpy as np
import glob
import os

WAV_DIR = "path/to/ESD/0011"
OUT_DIR = "backend/data/reference_audio"
os.makedirs(OUT_DIR, exist_ok=True)

EMOTIONS = ["Angry", "Happy", "Neutral", "Sad", "Surprise"]
TARGET_DURATION = 8.0 # seconds
SR = 22050

for emotion in EMOTIONS:
    wavs = sorted(glob.glob(f"{WAV_DIR}/{emotion}/*.wav"))
    combined = []
    total = 0.0
    for wav_path in wavs:
        audio, sr = sf.read(wav_path, dtype="float32")
        if audio.ndim > 1:
            audio = audio.mean(axis=1)
        combined.append(audio)
        total += len(audio) / SR
        if total >= TARGET_DURATION:
            break
    merged = np.concatenate(combined)
    out_path = os.path.join(OUT_DIR, f"{emotion.lower()}.wav")
    sf.write(out_path, merged, SR)
    print(f"{emotion}: {total:.1f}s saved")
```

This produced five reference files:

```
backend/data/reference_audio/
angry.wav      (~8 seconds, speaker 0011)
happy.wav      (~8 seconds, speaker 0011)
neutral.wav    (~8 seconds, speaker 0011)
sad.wav        (~8 seconds, speaker 0011)
surprise.wav   (~8 seconds, speaker 0011)
```

The **Fear** emotion detected by DistilRoBERTa maps to **surprise.wav** because both emotions share high-arousal acoustic characteristics — raised pitch and faster speech rate. **Disgust** maps to **neutral.wav** as the

closest available acoustic match.

1.2 GoEmotions Dataset

What it is

GoEmotions is a large-scale emotion-labelled text dataset released by Google Research in 2020. It contains **58,000 English Reddit comments** annotated with **27 distinct emotion categories** by crowdsourced human raters. It is the most granular publicly available emotion text dataset and is free to download under the Apache 2.0 licence.

How to get it

Available directly via Hugging Face Datasets:

```
from datasets import load_dataset
dataset = load_dataset("google-research-datasets/go_emotions")
```

Or download directly from: <https://github.com/google-research/google-research/tree/master/goemotions>

How we used it

We did not use GoEmotions directly in our training pipeline. It was used by the authors of the **j-hartmann/emotion-english-distilroberta-base** model to fine-tune DistilRoBERTa for six-category emotion classification. Our system benefits from this indirectly — by loading their pre-trained weights we inherit the emotion recognition capability trained on GoEmotions without needing to run the fine-tuning process ourselves.

2. Models

2.1 j-hartmann/emotion-english-distilroberta-base

What it is

This is a DistilRoBERTa transformer model fine-tuned by Jochen Hartmann on the GoEmotions dataset specifically for six-category English text emotion classification. DistilRoBERTa is a distilled (compressed) version of RoBERTa — itself an optimised variant of the BERT architecture — that achieves comparable accuracy at roughly half the inference time and memory footprint of the full model. The six categories it outputs are:

- **anger** → mapped to **Angry** in our system
- **disgust** → mapped to **Disgust** in our system
- **fear** → mapped to **Fear** in our system
- **joy** → mapped to **Happy** in our system
- **neutral** → mapped to **Neutral** in our system

- **sadness** → mapped to **Sad** in our system
- **surprise** → mapped to **Surprise** in our system

The model is approximately **250 MB** in size and runs efficiently on CPU, completing a single inference in under 150ms. It is freely available under a permissive licence for research and commercial use.

How to get it

It downloads automatically the first time it is used via the Hugging Face Transformers library. No manual download is required:

```
from transformers import pipeline

classifier = pipeline(
    "text-classification",
    model="j-hartmann/emotion-english-distilroberta-base",
    return_all_scores=False
)
```

On first run, the model downloads to `~/.cache/huggingface/hub/` and is cached there permanently for all subsequent uses. You can also browse the model card and download manually at:

<https://huggingface.co/j-hartmann/emotion-english-distilroberta-base>

How we used it

The model is loaded once at application startup inside `app/services/emotion.py` as a singleton inference pipeline, meaning it is initialised only once and reused for every request rather than being reloaded per request — which would be far too slow.

Every time a user submits text to the system, the raw text string is passed directly to the pipeline without any preprocessing:

```
result = classifier(text)[0]
emotion_label = result["label"] # e.g. "joy"
confidence = result["score"] # e.g. 0.87
```

The raw label returned by the model uses lowercase GoEmotions category names. These are normalised to our internal capitalised naming convention before being used downstream:

```
LABEL_MAP = {
    "joy": "Happy",
    "sadness": "Sad",
    "anger": "Angry",
    "fear": "Fear",
    "surprise": "Surprise",
    "neutral": "Neutral",
}
```

```
"disgust": "Disgust",  
}  
emotion = LABEL_MAP.get(emotion_label, "Neutral")
```

The normalised emotion label and its confidence score are then passed to the reference audio selector and synthesis pipeline, logged to the PostgreSQL database, and displayed on the Next.js frontend so the user can see which emotion was detected.

To prevent redundant transformer inference for repeated identical text inputs, the emotion detection result is wrapped in Python's `lru_cache` with a capacity of 256 entries. When the same sentence is submitted twice, the cached label and confidence are returned immediately without invoking the model again.

2.2 XTTS-v2 (Coqui AI)

What it is

XTTS-v2 is an end-to-end neural voice cloning model developed by Coqui AI and released in late 2023. It is a GPT-conditioned architecture trained on thousands of hours of multilingual expressive speech across 17 languages. Unlike two-stage TTS pipelines that require a separate acoustic model to generate mel spectrograms and a separate vocoder to convert those spectrograms to audio, XTTS-v2 performs both operations inside a single unified model, generating a full audio waveform at **24,000 Hz** directly from text and a reference audio conditioning input in a single forward pass.

The key capability that makes XTTS-v2 suitable for emotionally expressive synthesis is **reference audio conditioning**. You provide a short audio clip of 6–30 seconds from any speaker, and the model's encoder converts that clip into a conditioning latent vector — a compact high-dimensional numerical representation that captures the speaker's vocal identity, speaking style, pitch contour distribution, energy patterns, and emotional prosody. The GPT-based decoder then generates audio tokens that satisfy both the linguistic content of the target text and the acoustic profile encoded in the latent. The output sounds like the person in the reference clip saying the target text, inheriting their emotional character without that person ever having recorded those specific words.

The model weights are approximately **1.87 GB** in size. It is freely available under the Coqui Public Model Licence (CPML) for non-commercial use.

How to get it

****Automatic download via Coqui TTS library****

First install the library:

```
pip install coqui-tts
```

The licence agreement must be pre-accepted via environment variable before the model will download. In interactive terminals you would be prompted to type `y`, but in automated or server environments this prompt causes the process to hang indefinitely, so we bypass it:

```
export COQUI_TOS_AGREED=1
```

Then load the model — it downloads automatically on first use and caches locally:

```
import os
os.environ["COQUI_TOS_AGREED"] = "1" # must be set BEFORE importing TTS

from TTS.api import TTS

tts = TTS(
    model_name="tts_models/multilingual/multi-dataset/xtts_v2",
    progress_bar=True,
)
```

On first run the model (~1.87 GB) downloads to `~/.local/share/tts/tts_models--multilingual--multi-dataset--xtts_v2/` and is cached there. All subsequent loads read from this cache and take only a few seconds.

PyTorch 2.6 compatibility — critical patch

PyTorch 2.6 changed the default behaviour of `torch.load()` from `weights_only=False` to `weights_only=True`. The XTTS-v2 checkpoint uses an older serialisation format that is incompatible with this new default, causing the model to fail to load with a `pickle.UnpicklingError`. We fix this by patching `torch.load` before the TTS library is imported, which must happen at the very top of `xtts.py` before any TTS import:

```
import torch

# Patch must be applied BEFORE importing TTS
_original_torch_load = torch.load
def _patched_torch_load(f, *args, **kwargs):
    kwargs.setdefault("weights_only", False)
    return _original_torch_load(f, *args, **kwargs)
torch.load = _patched_torch_load

# Now safe to import
from TTS.api import TTS
```

If this patch is applied after `from TTS.api import TTS`, the TTS module has already been imported and the checkpoint loading has already been attempted — the patch will have no effect.

How we used it

Loading

The model is loaded once at application startup as a singleton inside `app/services/xtts.py`. The `.to()` call moves the model to GPU if one is available, falling back to CPU:

```
self.tts = TTS(
    model_name="tts_models/multilingual/multi-dataset/xtts_v2",
    progress_bar=False,
).to("cuda" if torch.cuda.is_available() else "cpu")
```

Reference audio download at startup

Because the five reference wav files are binary and cannot be pushed to Hugging Face Spaces via git (Hugging Face Spaces rejects binary files unless using Git LFS or their xet system), the files are uploaded to the Space via the web UI and then downloaded by the container at startup. If the files are not found locally, the service attempts to fetch them:

```
def _download_refs_if_missing(self):
    os.makedirs(_REF_DIR, exist_ok=True)
    BASE_URL = (
        "https://huggingface.co/spaces/Highfee/daneena_tts"
        "/resolve/main/data/reference_audio"
    )
    files = ["angry.wav", "happy.wav", "neutral.wav", "sad.wav", "surprise.wav"]
    for filename in files:
        out_path = os.path.join(_REF_DIR, filename)
        if not os.path.isfile(out_path):
            urllib.request.urlretrieve(f"{BASE_URL}/{filename}", out_path)
```

Synthesis

At inference time, the detected emotion label is used to look up the corresponding reference audio path, and both are passed to XTTS-v2:

```
wav = self.tts.tts(
    text=text,
    speaker_wav=ref_path,    # e.g. "data/reference_audio/happy.wav"
    language="en",
)
```

The model returns a Python list of float values representing the raw waveform at 24,000 Hz. This is converted to a NumPy float32 array and passed to the post-processing layer.

Post-processing

After XTTS-v2 generates the waveform, a lightweight post-processing step is applied to reinforce perceptual differences between emotion categories. This uses `librosa.effects.time_stretch` for tempo adjustment and a decibel-to-linear amplitude multiplication for loudness:

```
# Tempo
if abs(tempo_factor - 1.0) > 0.01:
    wav = librosa.effects.time_stretch(wav, rate=tempo_factor)

# Loudness
loudness_factor = 10 ** (loudness_db / 20.0)
wav = wav * loudness_factor

# Normalise to prevent clipping
peak = np.abs(wav).max()
if peak > 0:
    wav = wav * (0.95 / peak)
```

The per-emotion parameters are:

Emotion	Reference File	Tempo Factor	Loudness (dB)
Angry	angry.wav	1.10× faster	+5.0 dB
Happy	happy.wav	1.08× faster	+3.0 dB
Neutral	neutral.wav	1.00× unchanged	0.0 dB
Sad	sad.wav	0.82× slower	−3.0 dB
Surprise	surprise.wav	1.06× faster	+4.0 dB
Fear	surprise.wav	1.12× faster	+2.0 dB

The final normalised waveform is written to disk as a 16-bit PCM WAV file at 24,000 Hz using the `soundfile` library and served to the client via the audio endpoint.

Caching

Full synthesis results are cached using a second `lru_cache` with a capacity of 64 entries, keyed on the combination of input text, detected emotion, speed, pitch shift, and energy shift. When an identical request is submitted again, the cached audio array is returned immediately without invoking XTTS-v2 again — avoiding 400–500ms of neural inference for repeat requests.

3. Summary Table

Asset	Type	Source	Size	Used in Production
ESD (speaker 0011, English)	Dataset	GitHub — NUS/SUTD	~350 MB (English only)	✓ Reference audio source
GoEmotions	Dataset	GitHub — Google Research	~50 MB	✗ Used by DistilRoBERTa authors
j-hartmann/emotion-english-distilroberta-base	Model	Hugging Face Hub	~250 MB	✓ Emotion classification

Asset	Type	Source	Size	Used in Production
XTTS-v2	Model	Coqui TTS / Hugging Face	~1.87 GB	☒ Speech synthesis

4. Dependency Installation

```
# Core ML and audio
pip install coqui-tts
pip install transformers==4.37.2
pip install torch==2.1.0
pip install torchaudio==2.1.0
pip install torchvision==0.16.0
pip install librosa
pip install soundfile
```

5. First-Run Checklist

When deploying to a new environment, the following must complete successfully before the first user request can be served:

1. **COQUI_TOS_AGREED=1** must be set as an environment variable before the server starts — without it the XTTS-v2 download prompt causes the process to hang waiting for keyboard input
2. **DistilRoBERTa weights** download ~250 MB to `~/.cache/huggingface/hub/` on first import of the pipeline — requires internet access on first run only
3. **XTTS-v2 weights** download ~1.87 GB to `~/.local/share/tts/` on first `TTS()` call — requires internet access on first run only and sufficient disk space
4. **Reference audio files** — five wav files must exist in `data/reference_audio/` — the service will attempt to download them from the Hugging Face Space file store if not found locally, so internet access is also required if they are absent
5. **PyTorch patch** — the `torch.load` patch in `fastspeech.py` must be present and must appear before the `from TTS.api import TTS` import line — if missing the server will crash on startup with a `pickle.UnpicklingError`